

Week 12 Interim Progress Report

Project: Stock Challenge - Financial News Sentiment Analysis

Challenge: Week 12 - Improve Previous Project into Production-Grade Portfolio Piece

Report Date: February 17, 2026

Submission: Second Interim (Final Day)

1. Original Plan (Created Feb 11, 2026)

Based on my Gap Analysis, I identified 5 high-impact improvements to transform this project into a finance-sector ready portfolio piece.

Planned Tasks with Time Estimates

Task ID	Task Descriptio	Planned Time	Planned Complet
T1	Code Refactorin	3-4 hours	Feb 12
T2	Unit Testing -	2-3 hours	Feb 13
T3	CI/CD Pipeline	1-2 hours	Feb 14
T4	Interactive Das	4-5 hours	Feb 15
T5	Professional Do	2-3 hours	Feb 17

Total Planned Time: 12-17 hours across the week

2. Plan vs. Actual Progress

Progress Comparison Table

Task ID	Original Plan	Actual Progress	Status	Variance
T1	Create modular	Created 8 modul	[DONE] Complete	On time
T2	Write 5+ unit t	Wrote 35 unit t	[DONE] Complete	Exceeded plan
T3	Set up GitHub A	Created CI conf	Partial	Blocker encount
T4	Build Streamlit	Dashboard with	[DONE] Complete	On time
T5	Update document	README rewritte	[DONE] Complete	On time

Summary

- Completed: 4 out of 5 tasks fully completed

3. Completed Work Documentation

T1: Code Refactoring

What was done: Transformed flat notebook utilities into a professional Python package structure.

Before (Week 1):

```
notebooks/
  utils.py (398 lines, no type hints)
```

After (Week 12):

```
src/
__init__.py
config.py          # Dataclasses for configuration
data/
  loader.py        # Data loading with type hints
  preprocessor.py   # Data cleaning functions
analysis/
  sentiment.py      # TextBlob & VADER sentiment
  technical.py       # RSI, MACD, SMA, EMA, Bollinger
  statistics.py     # Correlation, Gini coefficient
visualization/
  plotting.py       # Reusable plotting functions
```

Code Sample - Type Hints & Dataclass:

```
@dataclass
class TechnicalAnalysisConfig:
    """Configuration for technical analysis calculations."""

    rsi_period: int = 14
    rsi_overbought: float = 70.0
    rsi_oversold: float = 30.0
    macd_fast: int = 12
    macd_slow: int = 26
    macd_signal: int = 9
    sma_periods: List[int] = field(default_factory=lambda: [20, 50, 200])

def calculate_rsi(prices: pd.Series, period: int = 14) -> pd.Series:
    """Calculate Relative Strength Index (RSI)."""
    delta = prices.diff()
    gain = delta.where(delta > 0, 0.0)
    loss = (-delta).where(delta < 0, 0.0)
    avg_gain = gain.rolling(window=period).mean()
    avg_loss = loss.rolling(window=period).mean()
    rs = avg_gain / avg_loss
    return 100 - (100 / (1 + rs))
```

Portfolio Value Enhancement:

- Demonstrates software engineering skills - Modular design shows I can build maintainable, scalable code

T2: Unit Testing

What was done: Created comprehensive test suite with 35 tests (7x the minimum requirement).

Test Results:

```
===== 35 passed in 13.91s =====

tests/test_config.py (12 tests):
- TestDataConfig: 3 tests
- TestTechnicalAnalysisConfig: 2 tests
- TestQualityReport: 4 tests
- TestStockMetrics: 1 test
- TestConstants: 2 tests

tests/test_analysis.py (23 tests):
- TestSentimentAnalysis: 10 tests
- TestTechnicalAnalysis: 5 tests
- TestStatistics: 8 tests
```

Sample Test Code:

```
class TestTechnicalAnalysis:
    """Tests for technical analysis functions."""

    def test_calculate_rsi(self, sample_prices):
        """Test RSI calculation."""
        rsi = calculate_rsi(sample_prices, 14)

        # RSI should be between 0 and 100
        valid_rsi = rsi.dropna()
        assert (valid_rsi >= 0).all()
        assert (valid_rsi <= 100).all()

    def test_calculate_macd(self, sample_prices):
        """Test MACD calculation."""
        macd, signal, histogram = calculate_macd(sample_prices)

        # Histogram should be MACD - Signal
        valid_idx = ~(macd.isna() | signal.isna())
        np.testing.assert_array_almost_equal(
            histogram[valid_idx],
            (macd - signal)[valid_idx],
            decimal=10
        )
```

Portfolio Value Enhancement:

- Demonstrates reliability - Finance sector values tested, reliable code

T3: CI/CD Pipeline (Partial)

What was done: Created GitHub Actions configuration for automated testing.

CI Configuration Created:

```

name: CI Pipeline

on:
  push:
    branches: [ main, development, task-* ]
  pull_request:
    branches: [ main, development ]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ['3.9', '3.10', '3.11']
    steps:
      - uses: actions/checkout@v4
      - name: Run tests with pytest
        run: pytest tests/ -v --cov=src

```

Current Status: Configuration created but push blocked (see Blockers section)

Portfolio Value Enhancement:

- Shows DevOps awareness - Understanding of automated quality checks

T4: Interactive Dashboard

What was done: Built Streamlit dashboard with 4 interactive views.

Dashboard Features:

1. Price Chart Tab - Candlestick chart with technical indicators (SMA, EMA, RSI, MACD, Bollinger Bands)

app/da

```

def create_price_chart(df, ticker, show_ma, show_bb):
    """Create interactive price chart with Plotly."""
    fig = make_subplots(rows=3, cols=1, ...)
    # Candlestick, RSI, MACD subplots
    return fig

def main():
    st.set_page_config(page_title="Stock Analysis Dashboard", ...)

    # Sidebar controls
    selected_stock = st.sidebar.selectbox("Select Stock", STOCK_TICKERS)

    # Key metrics display
    col1, col2, col3, col4, col5 = st.columns(5)
    with col1:
        st.metric("Latest Price", f"${latest_price:.2f}")

```

```
# Tabbed interface
tab1, tab2, tab3, tab4 = st.tabs([...])
```

How to Run:

```
streamlit run app/dashboard.py
```

Portfolio Value Enhancement:

- Demonstrates stakeholder communication - Non-technical users can explore data

T5: Professional Documentation

What was done: Rewrote README.md with all required sections per challenge guidelines.

README Sections Added:

- Business Problem (clearly articulated)

4. Current Blockers and Challenges

Blocker 1: GitHub OAuth Workflow Permission

Issue: Cannot push .github/workflows/ci.yml changes due to OAuth scope limitation.

Error Message:

```
! [remote rejected] task-3 -> task-3 (refusing to allow an OAuth App to
create or update workflow `.github/workflows/ci.yml` without `workflow` scope)
```

Impact: CI/CD pipeline configuration exists locally but not on GitHub.

Workaround Applied: Excluded workflow file from commit; other 33 files pushed successfully.

Resolution Plan:

1. Push workflow file manually via GitHub web interface, OR

Blocker 2: WeasyPrint System Dependencies

Issue: WeasyPrint PDF conversion requires system libraries (pango, cairo) that couldn't be installed due to Homebrew permission issues.

Workaround Applied: Created alternative PDF converter using fpdf2 (pure Python, no system dependencies).

Resolution Status: Resolved - PDFs generating successfully.

5. Revised Plan for Remaining Time

Since this is the final submission day (Feb 17), the revised plan focuses on:

Priority	Task	Time Needed	Status
1	Fix interim rep	1 hour	In Progress
2	Push CI workflo	30 min	Pending
3	Final report po	30 min	Pending

4	Verify all GitH	15 min	Pending
---	-----------------	--------	---------

6. Evidence of Work

GitHub Repository

URL: <https://github.com/Danielmituku/stock-challenge-week1/tree/task-3>

Commit History for Week 12

commit 2b92b6a - Week 12: Transform project into production-grade portfolio piece
33 files changed, 5721 insertions(+), 114 deletions(-)

Test Output Screenshot (Text)

```
===== test session starts =====
platform darwin -- Python 3.9.6, pytest-8.4.2
collected 35 items

tests/test_analysis.py::TestSentimentAnalysis::test_textblob_positive_sentiment PASSE
tests/test_analysis.py::TestSentimentAnalysis::test_vader_positive_sentiment PASSED
... (35 tests)
===== 35 passed in 13.91s =====
```

Files Created This Week

Category	Files
Source Code	`src/config.py`
Tests	`tests/test_con
Dashboard	`app/dashboard.
Config	`pyproject.toml
Documentation	`GAP_ANALYSIS.m
Utilities	`scripts/md_to_

7. Summary

Week 12 Accomplishments

- [DONE] Transformed notebook code into professional Python package (8 modules)

Portfolio Value Added

This project now demonstrates:

1. Software Engineering Best Practices - Modular code, type hints, dataclasses

Remaining Work

1. Resolve CI/CD OAuth blocker
-

Report Prepared By: Daniel Mituku

Date: February 17, 2026

GitHub: <https://github.com/DanielMituku/stock-challenge-week1>